

GoLLeM-v5: Small Language Models for the Glint Tiny-ML Leaderboard

Measurement, Method and Negative Results

Arek Słota

Kolektyw: Latarnik, Hart, Monter, Wartownik

Internal technical report — draft of September 26, 2026

Abstract

We report how we trained and evaluated GoLLeM-v5, a family of small decoder-only language models (16M–128M parameters) built to compete on the Glint Tiny-ML Leaderboard, whose score combines BLiMP, ARC-Easy and WikiText-2 byte perplexity with a size multiplier. Three findings dominate. First, *measurement*: leaderboard rows are self-reported under different protocols, so we reimplemented the canonical benchmark script and showed that our harness reproduces an independent maintainer measurement to the hundredth on ARC and exactly on WikiText (token perplexity 28.05 and byte perplexity 2.372 with the same 3.8605 bytes per token). On one protocol the gap between our best model (eff 75.81) and the strongest model we could measure (eff 78.26) is about 2.4 eff, not the 4.4 visible on the board, and it lies mostly in ARC (more than half), with BLiMP second. Second, *method*: every comparison was decided by rules written down before the result, on selection sets disjoint from the leaderboard test sets, with noise floors estimated from seed pairs. Third, *negative results*: data changes in the learning-rate tail and a WSD building pair stayed within seed noise; doubling model size raised quality but the size multiplier cancelled it in eff; training from scratch on FineWeb-Edu instead of our mixture and a deeper-narrower shape did not pass their pre-registered thresholds. We document the contamination, licensing and pipeline issues found along the way, the final 64M and 128M runs, and the levers we have not yet tested.

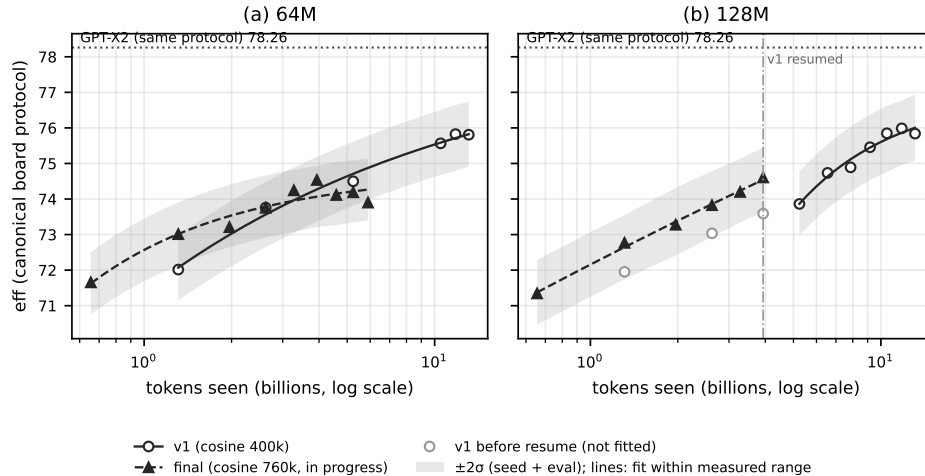


Figure 1: Efficiency (canonical board protocol) versus training tokens, 64M (a) and 128M (b). Circles: v1 runs (cosine over 400k steps); triangles: final runs (cosine over 760k steps, in progress; *not results*). Lines are fits of $y = a - bD^{-c}$ within the measured range only, not projections; bands are $\pm 2\sigma$. Dotted line: the best model measured on the same protocol (GPT-X2, 78.26). Resumptions and limits: Section 1.

Contents

1	Introduction	3
2	Measurement	4
2.1	The efficiency score	5
2.2	The canonical evaluation protocol	5
2.3	Why rows on the board are not directly comparable	6
2.4	Evaluation splits	7
3	Models and training	7
3.1	Architecture	7
3.2	Optimisation	7
3.3	Trainer versions and the data-sampling defect	8
3.4	Hardware and throughput	8
4	Data	9
4.1	ARC-MIX: composition	9
4.2	Contamination and the final pool	9
4.3	Licences	10
4.4	Experimental pools	11
4.5	Pipeline practice	11
5	Research method	11
5.1	Rules before results	11
5.2	Selection sets disjoint from the test	11
5.3	Noise floors	12
5.4	Paired intervals	12
5.5	Checkpoint branching	12
5.6	Scope statements	12
6	Experiments	12
6.1	Evaluation for decisions	12
6.2	Data changes in the cosine tail (rounds 1–4)	13
6.3	Scale: 64M versus 128M	14
6.4	Continuing from the flagship: the building pair (S12)	14
6.5	Training data from scratch (Z1)	14
6.6	Deeper shape at equal size (H4)	15
6.7	Stricter educational filter (Z4) — in progress	16
6.8	Summary	16
7	Final GoLLeM-v5 models	17
7.1	Setup	17
7.2	Evaluation protocol	17
7.3	Results	17
7.4	Training dynamics	18
8	Discussion	19
8.1	Why the levers did not move eff	19
8.2	What the top models do differently	19
8.3	Levers not yet tested	19
8.4	Limitations	20

9	How the work was done	20
9.1	Team	20
9.2	Verification from bytes	20
9.3	Errors we caught	20
9.4	What we would keep	21
	Acknowledgements and contributions	21
A	Reproducibility	22
A.1	Artefacts of the final v5 runs	22
A.2	Hardware and throughput	23
A.3	Safeguards in the launcher	23
B	Decontamination and data licences	23
B.1	What we protect	23
B.2	Two scans	24
B.3	What we found in our own corpus	24
B.4	Licences	24

1 Introduction

The Glint Tiny-ML Leaderboard ranks small language models by a single efficiency score. For a model with BLiMP accuracy B , ARC-Easy accuracy A (both in percent), WikiText-2 byte perplexity w and p parameters, the board computes

$$\text{WikiScore}(w) = 100 \left(1 - \frac{\ln w - \ln w_{\min}}{\ln 500 - \ln w_{\min}} \right), \quad w_{\min} = 1.86, \quad (1)$$

$$\text{mult}(p) = 1 + 0.5 \frac{\log_{10} p_{\max} - \log_{10} p}{\log_{10} p_{\max} - \log_{10} p_{\min}} \approx 1 + 0.042 \ln \frac{150\text{M}}{p}, \quad (2)$$

$$\text{eff} = \frac{B + A + \text{WikiScore}(w)}{3} \cdot \text{mult}(p), \quad (3)$$

with $p_{\max} = 150\text{M}$ and $p_{\min} = 10^3$ taken from the current board and WikiScore clipped to $[0, 100]$. We verified this formula line by line against the board source and reproduced displayed values to two decimals. The multiplier gives a 64M model a factor 1.0365 and a 128M model 1.0084: doubling parameters costs about 2.1 eff, which the extra quality has to repay.

Goal. The project goal set by the team lead was the first place in eff. Around it we set three working goals: (i) find out which training data actually trains better at this scale, (ii) build a catalogue of dataset quality with every entry stating what it was compared against, and (iii) keep written notes the team can learn from. This report is the third goal applied to the first two.

Where we stand. Measured with the canonical protocol (Section 2), our models score 74.33 (16M), 75.41 (32M), 75.81 (64M flagship, our reference) and 75.84 (128M). The best model we could measure on the same protocol, GPT-X2-125M, scores 78.26. The first row on the board reads 80.20, but board rows are self-reported under different evaluation settings (Section 2), so we use the single-protocol gap of about 2.4 eff as the real distance. Per axis, our BLiMP and wiki are mid-pack or better; ARC-Easy is lower by 3–8 points. On the 64M multiplier one ARC point is worth about 0.35 eff, so closing the gap needs roughly +7 ARC points at unchanged BLiMP and wiki.

Figure 1 at a glance. The figure shows where the final GoLLeM-v5 runs stand against the v1 runs of the same size at equal numbers of training tokens, on the leaderboard’s own protocol. Early in training the curves of the final runs track the v1 curves closely. At 40k steps the final runs were ahead by about one efficiency point (64M: 73.01 versus 72.02; 128M: 72.77 versus 71.95). The 128M final run stayed ahead (73.83 versus 73.04 at 80k, 74.61 versus 73.59 at 120k), while the 64M runs were level at 80k (73.75 versus 73.76) and the final 64M run was 0.31 behind at 160k (74.19 versus 74.50), when v1’s shorter cosine had its learning rate at about 69% of peak against 91% for the final run. Between 120k and 180k the final 64M run oscillates within about 0.6 points; the BLiMP part of that movement comes from a few rule-level categories flipping back and forth (Section 7.4). Two things cannot be read from the figure. First, the final value of a run: with a cosine schedule most of the gain comes when the learning rate decays at the end, and the final runs decay over 760k steps instead of 400k. Second, a ranking of recipes: points of one run are not independent, and the bands reflect measurement and seed noise estimated from a single seed pair. The 128M v1 run was resumed at 120k steps (dash-dotted vertical line), so its curve is fitted only after the resumption (steps $\geq 160k$); its 40k, 80k and 120k points (grey) are shown but not fitted, and at 1–3B tokens only point-to-point comparisons are available. The 64M v1 run was logged live without interruption from step 256k to 400k (largest gap between logged points 45s, the step never decreases); its history before 256k is unknown (its training log was lost), so panel (a) marks no resumption. Bands are $\pm 2\sigma$ with σ per axis and training stage (Section 5). With 3–7 points per run and a single cosine schedule the curve parameters are not identifiable (the asymptote and the exponent trade off against each other), so the fitted lines are descriptive only. The figure is a progress chart; the result of each run is its last checkpoint (Section 7).

Contributions.

- A reimplement of the canonical leaderboard protocol, checked against an independent maintainer measurement, and a per-axis comparison of top models on that single protocol (Section 2).
- A training and data pipeline for 32M–128M models with n -gram decontamination and, for the composition tests, a semantic gate, and the contamination and licence findings it produced (Sections 3, 4).
- A pre-registration style method for small-budget ablations: selection sets disjoint from test, seed-pair noise floors, and a written rule for every decision (Section 5).
- A set of controlled experiments, most of them negative, with confidence intervals (Section 6), and the final GoLLeM-v5 64M and 128M runs (Section 7).
- An account of how a small team of human and AI agents worked, including the errors we caught in our own work (Section 9).

2 Measurement

Everything in this report is judged by one number, the leaderboard’s *efficiency* score. Before any experiment we therefore pinned down exactly how that number is computed, reproduced it independently, and separated the data we are allowed to look at while choosing from the data we only report on.

2.1 The efficiency score

The leaderboard computes, for a model m with ARC-Easy accuracy A , BLiMP accuracy B (both in percent) and WikiText-2 byte perplexity w ,

$$\text{Score}(m) = \frac{1}{3}(A + B + W(w)), \quad (4)$$

$$W(w) = 100 \cdot \text{clip}_{[0,1]} \left(1 - \frac{\ln \min(w, 500) - \ln w_{\min}}{\ln w_{\max}^{\text{cap}} - \ln w_{\min}} \right), \quad (5)$$

$$\text{mult}(p) = 1 + 0.5 \cdot \text{clip}_{[0,1]} \left(\frac{\log_{10} p_{\max} - \log_{10} p}{\log_{10} p_{\max} - \log_{10} p_{\min}} \right), \quad (6)$$

$$\text{eff}(m) = \text{Score}(m) \cdot \text{mult}(p_m), \quad (7)$$

where p is the parameter count declared in the board entry, and $w_{\min}$, $w_{\max}^{\text{cap}}$, $p_{\min}$, $p_{\max}$ are taken over all models currently on the board.

The score is relative. Both normalisations depend on the board’s current extremes. At the revision we used, these were $w_{\min} = 1.86$, $w_{\max}$ capped at 500, $p_{\min} = 1$ K and $p_{\max} = 150$ M (76 entries with a parameter count, 78 with a WikiText value). Adding or removing a single model at either extreme changes every other model’s efficiency. All efficiencies in this report use these constants; they are not invariant across board revisions.

Consequences for our model sizes. With these constants the size multiplier is 1.0657 at 31.3 M parameters, 1.0365 at 62.9 M and 1.0084 at 122.8 M. Two consequences shaped our plans:

- At 64 M, one point of ARC-Easy or BLiMP is worth $1.0365/3 \approx 0.35$ efficiency points; WikiScore saturates near $w \approx 2$, where further perplexity gains buy little ($W(2.3718) = 95.66$).
- Doubling the model from 64 M to 128 M costs about 2.1 efficiency points through the multiplier alone ($73.14 \times (1.0365 - 1.0084)$), which the larger model has to earn back in quality. Our first 128 M model earned back exactly that and no more (Section 6).

Reproduction. Implementing the formula above against the board source reproduces our entries to the reported precision: the 64 M flagship ($A = 47.94$, $B = 75.83$, $w = 2.3718$) gives 75.81 and the 128 M model (51.94, 77.26, 2.2717) gives 75.84.

2.2 The canonical evaluation protocol

The leaderboard’s reference harness (`benchmark.py` of Glint-1.3) defines the three axes as follows; our harness implements the same choices:

- **BLiMP:** the 67 paradigms listed by exact configuration name, 1,000 pairs each (67,000 pairs). Each sentence is truncated to 256 tokens and scored by the sum of the log-probabilities of its tokens after the first (the first token is not scored); a pair counts as correct when the grammatical sentence scores strictly higher.
- **ARC-Easy:** test split (2,376 questions). For each answer option a the harness computes $\log p(qa) - \log p(q)$, where qa is the question q followed by a space and a , i.e. the log-likelihood of the answer given the bare question, without length normalisation, and takes the highest; the metric is raw accuracy.
- **WikiText-2:** the test split joined with spaces into one text, scored in consecutive windows of 256 predicted tokens without carried-over context; the result is token-level perplexity. Conversion to byte perplexity with the tokenizer’s bytes-per-token ratio is left to the submitter.
- **No explicit BOS token:** text is encoded with the tokenizer as is, including any tokens its own post-processor adds.

Table 1: Leading models and ours on one protocol (our harness, canonical choices of Section 2.2). Efficiency uses the board constants of Section 2.1 and each model’s declared parameter count. “Board” is the efficiency displayed on the leaderboard.

Model	ARC-E	BLiMP	Wiki (byte ppl)	Eff.	Board
GPT-X2-125M	55.72	80.28	2.199	78.26	80.06
Supra-50M-Base	50.93	76.73	2.426	77.62	76.83
JugnuLM-110M-R2+	54.08	77.73	2.256	77.12	80.20
JugnuLM-53M	50.88	74.86	2.519	76.62	79.29
Haidass-143M-v1	56.23	76.50	2.295	76.48	79.83
GoLLeM-v5 128M (ours)	51.94	77.26	2.2717	75.84	–
GoLLeM-v5 64M (ours)	47.94	75.83	2.3718	75.81	77.51

For our 12,288-token BPE tokenizer the WikiText-2 test text has 1,292,008 UTF-8 bytes and 334,674 tokens, i.e. 3.8605 bytes per token. The board maintainer re-measured our 64 M checkpoint independently and obtained the same token perplexity (28.0517), the same ARC-Easy accuracy (47.94) and, with the same byte count, the same byte perplexity (2.372). (Their first pass counted bytes by decoding each token separately, 3.864 bytes per token, and gave 2.370.)

2.3 Why rows on the board are not directly comparable

Submissions are self-reported, and several top entries were measured with the `lm-evaluation-harness` rather than the reference script. The two protocols differ in ways that move the score by points, not decimals:

- **How the start of a BLiMP sentence is scored.** The canonical script does not score the first token. Prepending the tokenizer’s document separator as a BOS token raised our 64 M BLiMP from 75.83 to 78.56 (+2.73) and that of JugnuLM-53M by +2.05; an `lm-evaluation-harness` multiple-choice run, which replaces the empty context by a space token and puts a space before the sentence, gives 77.84.
- **Context length for WikiText.** Longer or sliding windows lower perplexity relative to 256-token windows with reset.

We therefore re-measured the leading models with *our* harness, using their pinned public checkpoints (remote code reviewed at the pinned revision before loading). Table 1 lists the result. On a single protocol the best model, GPT-X2-125M, scores 78.26 and our 64 M model 75.81, a gap of 2.4 efficiency points, against 4.4 on the board as displayed. For the models we re-measured, the displayed score exceeds the single-protocol score by between -0.8 and $+3.4$ points. Robustness of this comparison was checked only for one pair of models (the gap to JugnuLM-53M stayed between 0.5 and 0.8 points across all four protocol variants we tried), so it should be read as indicative.

Our own error: board PR #78. Our initial board submission reported BLiMP 77.84 and WikiText 2.016 (efficiency 77.51, rank 7). The WikiText value came from converting the correct token perplexity with a wrong bytes-per-token factor (4.755 instead of 3.8605; $28.0517^{1/4.755} = 2.016$). The BLiMP value came from the `lm-evaluation-harness` multiple-choice variant described above, which scores the first word of each sentence; we reproduced 77.84 to the hundredth on the same checkpoint, while the canonical protocol gives 75.83 (the maintainer’s independent run, on 73,000 pairs, gave 75.99). On the canonical protocol the checkpoint scores efficiency 75.81 (rank 14). We acknowledged the WikiText error in the pull-request thread the next day, and the maintainer posted the same correction on our model repository; at the time of writing the board still shows the original row.

role	layers $\times$ width $\times$ heads	parameters	size mult.	used in
32M reference shape	$15 \times 384 \times 6$	31 297 550	1.0657	Z1, Z4 (§6.5, §6.7)
32M deep-narrow	$22 \times 320 \times 5$	31 004 117	1.0660	H4 (§6.6)
64M (flagship)	$14 \times 576 \times 9$	62 867 021	1.03646	v1, rounds 1–4, S12, final
128M	$16 \times 768 \times 12$	122 759 951	1.0084	v1_128m, final

Table 2: Model shapes. Counts from the r6 GPT class (tied embeddings counted once). Deep variants were prepared for the final runs but not trained after H4 failed: 64M $33 \times 384 \times 6$ (63 191 840 parameters) and 128M $37 \times 512 \times 8$ (122 840 228).

2.4 Evaluation splits

Decisions are taken only on selection sets disjoint from the leaderboard’s test sets (ARC-Easy validation and train questions without any question found in a compared training pool, WikiText-2 validation without 8 articles present in our corpus, and a fixed half of BLiMP); they are defined in Section 5. Test-set numbers never pick anything: for a planned run the result is its final checkpoint, and test-set numbers of earlier checkpoints of a run whose recipe is frozen are published only as a progress table labelled “training in progress; not a result”.

3 Models and training

3.1 Architecture

All GoLLeM-v5 models are decoder-only transformers built by one class, `GPT`, of the r6 trainer. The components are standard in current small models:

- **Normalisation:** pre-norm RMSNorm, computed in fp32 ($\epsilon = 10^{-6}$).
- **Positions:** parameter-free rotary embeddings on q and k with base $\theta = 10^5$ (the value used by the two strongest board entries); no learned position table.
- **Feed-forward:** SwiGLU, $\text{down}(\text{silu}(\text{gate}(x)) \odot \text{up}(x))$, hidden width $2.667d$ (parameter parity with a $4d$ GELU MLP).
- **Attention:** QK-norm (per-head RMSNorm on q and k) and value residuals (ResFormer: $v_\ell \leftarrow v_\ell + \lambda_\ell v_0$, λ_ℓ initialised at 0).
- **Embeddings:** input and output embeddings are tied; head dimension 64 in every shape.
- **Tokenizer:** byte-level BPE with 12 288 tokens (end-of-document id 12 285), context 1024 tokens. On WikiText-2 test it produces 334 674 tokens for 1 292 008 bytes, i.e. 3.8605 bytes per token. Decoding is lossless: $\text{decode}(\text{encode}(t)) = t$ for 1 000 of 1 000 FineWeb-Edu documents.

Table 2 lists every shape trained in this campaign. Parameter counts are obtained by instantiating the class, not from a formula; the board’s size multiplier is a function of this count (Section 2).

Earlier models. Two smaller models precede this recipe. The 32M legacy model (31 601 664 parameters) was trained with AdamW, batch 64, 244 000 steps, LayerNorm and learned positions; on the board protocol it scores ARC-E 44.44, BLiMP 73.48, WikiText-2 byte perplexity 2.5386, eff 75.41. The 16M legacy model scores 40.91, 70.08, 2.6746 and eff 74.33. We do not describe their recipes further; they are shown for scale only.

3.2 Optimisation

The recipe is the one of the 64M flagship `v1_muon`; we read it back from the optimizer state stored in its last checkpoint rather than from launch notes.

- **Muon** for all two-dimensional hidden weights (attention and MLP matrices): momentum 0.95 with Nesterov, five quintic Newton–Schulz steps in bf16, no weight decay, peak learning rate 0.02.
- **AdamW** for embeddings, norm gains and other non-matrix parameters: $\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$, weight decay 0.1, peak learning rate $6 \cdot 10^{-4}$.
- One schedule drives both optimisers (Muon uses the fixed multiplier $0.02/6 \cdot 10^{-4} = 33.3$): 2000 linear warm-up steps, then cosine decay to $6 \cdot 10^{-5}$ at the last planned step.
- `torch.compile`, bf16 autocast, gradient clipping at 1.0, and batches of $32 \times 1024 = 32\,768$ tokens per step.

The 128M reference `v1_128m` uses the same recipe (400 000 steps, 13.1B tokens).

3.3 Trainer versions and the data-sampling defect

The trainer went through three versions during the campaign. Only the way training windows are drawn changes; model and optimiser code are identical.

v1 (up to r4) draws each batch as 32 random window starts *with replacement* (`torch.randint`).

At 1.4 passes over a corpus this covers only $1 - e^{-1.4} \approx 75\%$ of the windows. Worse, on `--resume` the window generator was re-seeded, so a resumed run re-drew exactly the windows of its first steps. The defect was found on 25 September by reading the RNG loop, after a “training floor” of 0.49 eff between two runs on the same data turned out to be fresh versus repeated data rather than noise (§6.2). `v1_128m` was resumed at step 120 000 with this trainer, so the windows of its steps 0–120k were most likely drawn twice; the flagship `v1_muon` ran without interruption from step 256k to 400k (live tracker log); its history before 256k is unknown (its training log was lost).

r5 fast-forwards the generator on resume (a resumed run draws the same windows as an uninterrupted one; tested to $\Delta\text{loss} = 0.0$) and adds `--rewarm-steps` and a WSD schedule for branch experiments.

r6 adds `--sampler epoch`: every epoch is a permutation of non-overlapping windows on a grid shifted by a per-epoch random offset, and step s takes windows $32s, \dots, 32s+31$ of the concatenated epoch stream. The window sequence is a pure function of the step, so resume is exact without fast-forwarding, and each window is seen once per epoch. With `--sampler random` r6 reproduces r5 bit for bit. Z1, Z4, H4 and the final runs all use r6 with the epoch sampler.

r6 writes a checkpoint only when $(s+1) \bmod \text{ckpt_every} = 0$ (line 602). The final runs therefore use step counts that are multiples of the checkpoint interval, so that the last step always has a checkpoint (Section 7).

3.4 Hardware and throughput

Training ran on single RTX 5090 pods; evaluation on a local RX 7900 XTX (ROCm). Table 3 gives the measured throughput of r6 by shape on one pod.

shape	tokens/s	hours for 25B tokens
32M 15×384	363 000	–
32M 22×320 (H4)	282 179	–
64M 14×576	222 384	31.2
64M 33×384	172 721	40.2
128M 16×768	140 502	49.4
128M 37×512	105 843	65.6

Table 3: Throughput by shape (Ryzen 9950X pod, RTX 5090). At equal parameter count the deep-narrow shapes have 22–25% lower throughput (28–33% longer training time).

Component	Tokens
minimal-en-corpus-5b (base)	5 396 605 407
FineWeb-Edu (minus 42 846 overlapping documents)	2 756 248 149
OpenStax, 58 CC BY 4.0 books $\times 4$	140 742 204
ARC-relevance extra copies (gold +2, related +1)	1 123 440 072
ARC-MIX (<code>train.bin</code> , SHA-256 <code>e87f594f</code>)	9 417 035 832
after test/validation scan (4 362 documents removed)	9 401 283 628
after the CC BY-NC-SA marker rule (1 175 documents removed)	9 391 706 576

Table 4: ARC-MIX composition and the two removal passes applied to the final v5 training pool.

4 Data

All models in this report are trained on token streams of a single BPE tokenizer with a vocabulary of 12 288 entries. Streams are stored as flat little-endian `uint16` arrays; documents are separated by the end-of-text token (id 12 285). The same held-out validation stream (`val.bin`, SHA-256 prefix `8253a108`) is used for all runs.

4.1 ARC-MIX: composition

ARC-MIX is the corpus behind the published GoLLeM-v5 32M–128M models and the final v5 runs. The 16M model used the expanded 8.29B-token corpus (step 2 below). It is built in three steps. The token counts below come from the merge scripts and build logs and add up exactly to the packaged total (Table 4).

1. **Base corpus.** SlayerLab/`minimal-en-corpus-5b`, a public multi-source English mixture: 5 396 605 407 tokens.
2. **Expansion.** FineWeb-Edu documents were appended, excluding 42 846 documents that overlap the base corpus (57 241 804 tokens skipped). This adds 2 756 248 149 tokens. We also appended 58 English OpenStax textbooks licensed CC BY 4.0 (35 185 551 tokens, each book stored as one document). The books are repeated four times, giving 140 742 204 tokens. The expanded corpus has 8 293 595 760 tokens.
3. **ARC-relevance upweighting.** 2 384 000 FineWeb-Edu documents were scored with a keyword heuristic (`arc_score`; domain word lists for biology, physics, chemistry and earth/space science, plus explanation and pedagogy cues, minus shop/boilerplate and research-jargon cues). Documents with score ≥ 4 (“gold”, 301 650 documents) received two extra copies. Documents with score = 3 (“related”, 184 484) received one extra copy. The 3 184 documents dropped by the build-time decontamination were excluded. The extra copies add 1 123 440 072 tokens.

The heuristic is topical. It does not retrieve ARC questions and treats all ARC splits the same way. Selection-time evaluation therefore uses ARC-Easy validation and train questions after decontamination (Section 5).

4.2 Contamination and the final pool

The build-time decontamination of the base corpus and of ARC-MIX used 13-gram shingles against the benchmark test sets. When we later scanned against the board’s test sets, together with a normalized 13-gram index and exact matching for short questions (details in Appendix B), we found:

- seven WikiText-2 *test* articles present at 66–99% coverage;
- 30 ARC-Easy and 13 ARC-Challenge test items, two of which are verbatim question copies.

The same seven articles, with identical coverage, and both ARC-Challenge copies are in the public base corpus. The contamination therefore comes from the base, not from our additions. Our additions contributed 7 ARC fact sentences.

These hits do not inflate our reported numbers beyond noise:

- On the seven articles, the ratio of byte perplexity on the contaminated subset to the rest is 1.021–1.028 for our 16M, 32M and 64M models. GPT-2 small, which was not trained on Wikipedia, gives 1.021. This is consistent with the articles being about 2% harder, with no memorization signal.
- Removing the 30 affected ARC-Easy items changes accuracy by at most 0.2 percentage points.

The measurement protocol is described in Section 2.

For the composition tests (Section 6) we added a semantic gate: MiniLM sentence embeddings (pinned revision 1110a243). The gate flags a document when its cosine similarity to a protected benchmark item is ≥ 0.95 and a lexical check confirms the match. Flagged questions are removed from the selection axis; the documents stay in the pool (the gate measures, it does not filter). Before its output is accepted on a machine, the gate must pass an acceptance test. The test runs a fixed package of 1 000 documents, and the output must be byte-identical to the reference machine. The Xeon training host reproduced the reference output exactly (0e977684). It was too slow, however, to scan the comparison pool C before the final runs: under 175 documents/s, CPU-bound, with the GPU idle. The Z4-versus-C axis was therefore not frozen in time.

For the final v5 runs the pool is ARC-MIX with every document removed that is hit by the validation-and-test scan (WikiText-2 test/validation, ARC validation/test) or by the licence rule below. Removal is in place and keeps document order; nothing is resampled. The compaction script asserts:

- contiguity of the document index;
- the exact number of removed documents ($5\,537 = 4\,362$ from the scan + $1\,175$ from the licence rule);
- the exact token count ($9\,391\,706\,576$) before the file is truncated.

The pool was built independently on both training machines from the same inputs. Both copies have SHA-256 `ecfd0a40`. The launcher refuses to start unless both the manifest and the file hash match.

4.3 Licences

The OpenStax source on the Hugging Face Hub (`crumb/openstax-text`) declares CC BY 4.0 for the whole collection. Checking each title’s own text showed ten titles under CC BY-NC-SA 4.0 and eleven non-English titles. Both groups were excluded at build time, which leaves the 58 books above.

While preparing the attribution file we found figure and table captions reading “Copyright Rice University, OpenStax, under CC BY-NC-SA 4.0 license” inside three of the 58 CC BY books (Organizational Behavior: 114 captions; Introduction to Business: 2; Principles of Management: 2). We then applied a conservative rule to the final pool: *a document whose text contains the marker “CC BY-NC-SA” is removed*. The rule matches the hyphenated form after decoding and accepts whitespace or line breaks between the parts. It does not match the unhyphenated spelling or the spelled-out licence name. It removed 1 175 documents (1 936 mentions, 9 577 052 tokens, 0.10% of the pool):

- 12 are the four copies of the three books;
- 1 163 are web pages, mostly open-educational-resource listings, LibreTexts pages and photo credits.

Some of the removed pages only *mention* the licence. The rule removes them anyway, by design.

The final models therefore list 55 OpenStax titles in their attribution file. Earlier published models and the public ARC-MIX files are unchanged, and the rule applies to all future training pools.

4.4 Experimental pools

The from-scratch composition tests (Section 6) used pools built with the same tooling and recorded in the same way (script hash, input hashes, pool hash):

- **Z:** FineWeb-Edu with quality score ≥ 3 from three samples (FineWeb-Edu part of the base corpus, `sample-100BT` and `CC-MAIN-2024-30`), 1 237 documents removed by the scan; pool `877f80d1`.
- **C:** the ARC-MIX control after the scan (4 362 documents removed); pool `27bae5ad`.
- **Z4:** FineWeb-Edu with score ≥ 4 ; pool `d8d49b35`. Relative to Z, this filter concentrates ARC-like quiz content: $13.3\times$ more distinct ARC train question IDs and $3.4\times$ more ARC test IDs per billion tokens. Its results are therefore read only on an evaluation axis with those items removed.

4.5 Pipeline practice

- **Hashes everywhere.** Every pool, script and input is identified by SHA-256. A local file is deleted only after its hash equals the LFS hash on the Hub.
- **Composition from build logs.** The composition of a pool is read from build logs and manifests, not from the build script. Once a docstring described a blend that the logs contradicted, and a claim based on it had to be retracted.
- **Resource limits measured.** Parallelism is set from the container CPU quota, not from the visible core count. On our pods the quota was about 13.6 cores for 32–128 visible cores. Free disk is checked before every job.
- **Gate before execution.** Every script that writes training data passes an independent review before it runs. The review identifies the script by hash.

5 Research method

With a few GPUs and hours per experiment, most interventions we could afford move off by less than one point. At that scale the main risk is not a wrong model but a wrong conclusion: selecting on noise, on the test set, or on a rule adjusted after seeing the numbers. We therefore fixed the following method and applied it to every decision in Section 6.

5.1 Rules before results

Each experiment had a written decision rule posted before its first evaluated checkpoint: which checkpoint counts (the last step of the planned schedule, never the best one), which axes are compared, and a numerical threshold for *every* axis, including the ones we did not expect to move. Where a rule had a gap, the gap resolved to the status quo. Reallocating compute after a result was allowed if stated openly; changing a verdict rule after a result was not. Where a rule was written after some descriptive points had been seen, we say so in the text.

5.2 Selection sets disjoint from the test

Leaderboard test sets are used only to report final numbers and, for the final runs, a public progress table of intermediate checkpoints labelled as such; they never select anything. Decisions use a *selection axis*:

- ARC: ARC-Easy validation plus train questions ($\approx 2.8k$ items), minus every question whose text was found in any compared training pool by n -gram or, where required, semantic matching; the exclusion list is frozen before the first evaluated checkpoint;
- wiki: WikiText-2 validation, minus the eight articles that our corpus contains as near-copies;
- BLiMP: one half of the pairs, the other half kept out of selection.

Canonical test numbers for candidate checkpoints were computed only after a decision, and written to files rather than shown on screen (one violation of the latter is disclosed in Section 9).

5.3 Noise floors

The quantity that decides whether an effect is real is the seed-to-seed difference of the same recipe. We estimated it from seed pairs at each scale: at 64M, two seeds of the same data differed by 0.28 eff; at 32M after 150k steps the pair differed by 0.03 ARC points, 0.35 BLiMP points and 0.09 eff. Thresholds were set as $\max(\text{floor}, 2|\Delta_{\text{seeds}}|)$ with a floor fixed in advance (e.g. 0.6 eff), because a single pair can land close by chance. Under a normal approximation with seed standard deviation ≈ 0.30 eff, a threshold of 0.30 would declare a false winner about 41% of the time, 0.6 about 10%.

For the learning curves of Figure 1 the band of each point uses $\sigma = \sqrt{\sigma_{\text{seed}}^2 + \text{SE}_{\text{eval}}^2}$, with σ_{seed} from the round-4 seed pair and SE_{eval} from a paired bootstrap over items of that checkpoint, so the band is wider early in training, where evaluation noise dominates (about 1 ARC point, i.e. ≈ 0.35 eff).

5.4 Paired intervals

Axis differences are reported with paired bootstrap intervals over items (same questions, same BLiMP pairs, same wiki windows in both models; seed 1337). A difference counts only if the rule’s threshold is met; an interval excluding zero alone is not enough.

5.5 Checkpoint branching

For data questions late in training we branched from a shared checkpoint and changed only the data in the branch (runbook RB4). This isolates the data effect from the early-training trajectory but, as Section 6 shows, measures the effect only at a small learning rate.

5.6 Scope statements

Every verdict carries its scope, e.g. “composition at $\approx 4.9B$ tokens on a 32M model”. Transfers to other sizes or budgets are marked as inference.

6 Experiments

All experiments follow the protocol of Section 5: the decision rule, its thresholds and the evaluation axis are fixed *before* the first number of an arm is seen, the choice between arms is made on selection splits, and the board test is used only to report final numbers (and, for the final runs, a public progress table of intermediate checkpoints labelled as such); it never selects anything. We report every experiment, including the ones that failed; most of them did.

6.1 Evaluation for decisions

Selection axis. Decisions use ARC-Easy validation (and, from the building pair on, ARC-Easy train), WikiText-2 validation and a fixed half of BLiMP (500 pairs per phenomenon, seed

round	arm (data of steps 320k–400k)	ARC-E	BLiMP	wiki bppl	eff
–	flagship v1_muon (arcmix, trainer v1)	47.94	75.83	2.3718	75.81
1	control: arcmix, resumed	47.69	76.29	2.3717	75.88
1	fork-B: arcmix + FineWeb-Edu (faulty build)	46.34	77.20	2.3982	75.66
3	A: arcmix + FineWeb-Edu 45:55 (fixed build)	47.31	76.22	2.3762	75.71
3	B: arcmix, QA documents $\times 2$	47.26	75.90	2.3712	75.60
4	anchor: arcmix, QA $\times 1$ (2.6B pool)	47.73	75.25	2.3732	75.53
4	B, seed 1338	46.80	75.57	2.3722	75.32

Table 5: Cosine-tail data rounds, board protocol, step 400 000 (ARC-E test 2 376, BLiMP 67 000, WikiText-2 test).

1337). Items that occur in the training data of any compared arm are removed before the first evaluation: 8 WikiText-2 validation articles that arcmix contains as near-copies ($\geq 10\%$ 13-gram coverage), a fixed list of 5 ARC questions, and every ARC question hit by the n-gram scan of the arms’ pools. The ARC-Easy axis therefore changed with the compared pools: 566 questions (validation only), 2 791 (validation+train, building pair), 2 783 (ZUC, experiments Z1 and H4) and 2 295 (ZUCU4, pending). ARC-Challenge validation is reported for direction only; at 32–64M it sits at chance level (20–26% in all runs of Tables 7–8).

Intervals. Arms are compared with a paired bootstrap over items (ARC per question, BLiMP per pair, wiki per 256-token chunk; $B = 5000$), recomputing eff in every resample. These intervals capture evaluation noise only. Training noise is measured separately with seed pairs: two runs that differ only in the seed differ by ± 0.3 eff at the end of a 64M run (four checkpoints, round 4) and by 0.09 eff at the end of a 32M run (Z1). A single seed pair is a weak estimate, so every rule that uses it also has a floor (0.6 eff, i.e. about two standard deviations of the round-4 seed noise).

6.2 Data changes in the cosine tail (rounds 1–4)

The first question was whether the last fifth of the 64M run can be improved by changing only its data. Every arm branches from the flagship checkpoint at step 320 000 and finishes the same cosine schedule at 400 000. Table 5 lists all arms on the board protocol at step 400 000.

Round 1 was confounded, and we found it by reading the data rather than the metrics: the fork-B blend took only the first 1.15B tokens of arcmix (no QA or chat documents), used document separator 12 287 instead of 12 285, and its 2.55B-token pool produced a different repetition regime from the 9.42B-token control. We report it only as “build fork-B versus control”: $\Delta\text{eff} = -0.22 [-0.64, +0.21]$, with ARC $-1.35 [-2.57, -0.13]$, BLiMP $+0.91 [+0.70, +1.10]$ and wiki worse by 0.027. The same round produced an apparent training floor of 0.49 eff between the flagship and the control on identical data; it is attributed to the trainer defect of Section 3.3, not to noise: on resumption the old sampler re-drew the windows of steps 0–80k. The control was resumed; the flagship ran without interruption from step 256k, so its compared tail (steps 320k–400k) was not resumed, and its history before 256k is unknown (its training log was lost).

Round 3 compared a correctly built FineWeb-Edu blend (A) with doubled QA documents (B). Across four checkpoints the difference in eff changed sign ($+0.46, -0.13, -0.02, +0.11$ at 340k–400k; at 400k $+0.11 [-0.30, +0.51]$). Stable in every checkpoint: FineWeb-Edu makes WikiText worse; BLiMP slightly better within noise. Compared with fork-B, the fixed build removed most of fork-B’s BLiMP gain and wiki loss, so the round-1 pattern was an effect of the faulty build, not of the educational data.

Round 4 added an anchor (QA once) and a second seed of arm B. The seed pair differs by +0.26, −0.34, −0.32 and −0.28 eff at 340k, 360k, 380k and 400k; QA×2 and FineWeb-Edu are both inside this noise relative to the anchor, and FineWeb-Edu makes wiki worse in 4 of 4 checkpoints.

Conclusion. Changing the data of the last 20% of a cosine run moves eff by less than the seed noise (≈ 0.3 – 0.5). The gap to the top of the board (about 2.4 eff on our harness) cannot be closed there.

6.3 Scale: 64M versus 128M

A 128M model (Table 2) was trained with the flagship recipe on the same data. At matched step 160 000 it is better on ARC-E and wiki (ARC-E 48.27 versus 44.99, byte perplexity 2.3471 versus 2.4436) and level on BLiMP (75.63 versus 75.52), but the size multiplier (1.0084 versus 1.03646) takes more than the gain, and the 64M model leads by 0.64 eff (74.50 versus 73.86). The 128M run closes the gap only at the end:

	step	ARC-E	BLiMP	wiki bppl	eff
	200k	49.62	76.78	2.3344	74.74
	240k	50.00	76.67	2.3094	74.89
	280k	51.14	77.10	2.2950	75.46
	320k	52.06	77.24	2.2812	75.85
	360k	52.53	77.12	2.2731	75.99
	400k (final)	51.94	77.26	2.2717	75.84

Table 6: v1_128m, board protocol. The reported result is the last planned step; 360k is shown only as trajectory.

At 400k the 128M model and the 64M flagship are tied (75.84 versus 75.81): at this recipe and token budget, doubling the model adds about as much quality as the multiplier removes.

6.4 Continuing from the flagship: the building pair (S12)

The second strategy extends the finished flagship in segments. Both arms restart from v1_muon step 400 000 with a warm-up-stable-decay schedule (re-warm 2 000 steps to $3 \cdot 10^{-4}$, constant, decay over the last 6.7% of 600 000 steps; trainer r5). Arm A continues on arcmix (2.62B-token pool), arm B on arcmix + FineWeb-Edu 45:55 (2.55B). A guard written in advance (A’s probe at 424k must keep wiki-val ≤ 2.3785 and half-BLiMP ≥ 75.57) did not fire (2.3737 and 75.64).

- Segment 1, probes at 424k: B−A $\Delta\text{eff} = +0.14$ $[-0.78, +1.05]$, with BLiMP +1.17 $[+0.88, +1.48]$ and wiki +0.0079 (worse).
- Segment 2, probes at 444k (new axis, 2 791 ARC questions, threshold 0.73 fixed before): B2−A −0.24 $[-0.68, +0.20]$. The BLiMP gain of segment 1 did not repeat (−0.12); the wiki loss did (+0.0061).
- H3, decaying to 0 instead of $6 \cdot 10^{-5}$: ARC-E −1.94 $[-3.71, -0.18]$, ARC-C −2.0, BLiMP +0.45, wiki −0.0038, $\Delta\text{eff} = -0.51$. Rejected.

Both segments are ties; the pair was stopped after segment 2. FineWeb-Edu made WikiText worse in every measurement of rounds 1–4 and S12.

6.5 Training data from scratch (Z1)

Question and design. Rounds 1–4 changed data only at the end. Z1 asks whether a different corpus helps when used from the first step, as in DataDecide (where FineWeb-Edu beats general web mixtures by 7–11 points of ARC). Three 32M runs ($15 \times 384 \times 6$, trainer r6, 150 000 steps =

step	Z, seed 1337			Z, seed 1338			C (arcmix)		
	ARC	BLiMP	bppl	ARC	BLiMP	bppl	ARC	BLiMP	bppl
30k	39.02	70.87	2.8367	40.53	69.77	2.8405	38.38	70.46	2.6786
60k	42.22	72.08	2.7566	41.29	71.41	2.7618	39.92	72.07	2.6063
90k	42.11	72.22	2.7155	41.79	71.69	2.7163	40.82	73.07	2.5617
120k	43.12	72.47	2.6768	42.58	71.87	2.6894	40.71	72.89	2.5380
150k	42.90	72.64	2.6701	42.87	72.99	2.6808	41.93	73.35	2.5153
eff (150k)	74.27			74.36			74.56		

Table 7: Z1 on the selection axis (ARC-E validation+train, 2783 questions; half BLiMP; WikiText-2 validation without 8 articles). eff with multiplier 1.0657.

4.915B tokens, identical flags): Z = FineWeb-Edu from three samples (two filtered at int score ≥ 3 , one through the base-corpus pipeline; pool 5.137B tokens) with seeds 1337 and 1338, and C = our arcmix, seed 1337. Evaluation on the ZUC axis (ARC-E 2783). The rule (written before any checkpoint): Z wins if ARC improves by more than 2 points with an interval excluding 0 *and* eff improves *and* BLiMP loses at most 1.0 *and* wiki worsens by at most 0.05. A second rule decides the corpus of the final runs: the arm with higher eff on the selection axis (mean of Z seeds), if the gap exceeds $\max(0.6; 2|\Delta_{\text{seed}}|)$; otherwise arcmix stays.

Result. At the final step, Z–C is: ARC +0.97 [−0.65, +2.55] and +0.94 [−0.65, +2.59] (seeds 1337 and 1338); BLiMP −0.71 [−1.17, −0.26] and −0.36 [−0.82, +0.10]; wiki +0.155 and +0.166 byte perplexity (worse, both intervals far from 0); eff −0.28 and −0.19 (the bootstrap scores eff with the 64M multiplier; sign and verdict are unchanged). **Z fails the rule** on ARC, eff and wiki. The two Z seeds agree closely (ARC 0.03, eff 0.09 apart), so the verdict is not seed-sensitive. For the corpus choice, $\bar{Z} - C = 74.315 - 74.558 = -0.24$, inside the 0.6 floor: a tie, and the final runs keep arcmix. On the board test (computed afterwards, descriptive only) the picture is the same: Z 74.64 and 74.55, C 74.78.

Reading. At 32M and 4.9B tokens, pure FineWeb-Edu gains about one point of ARC over our arcmix, within noise, and pays with clearly worse WikiText and slightly lower BLiMP. The DataDecide gap does not transfer because our control is not a general web mixture: arcmix already contains educational and QA material [inference]. The ARC difference between Z and C lies between +0.6 and +2.4 at every checkpoint and does not grow with training (Table 7).

6.6 Deeper shape at equal size (H4)

Question and design. BLiMP grows with size and depth together in published small-model results, and BLiMP counts as much as ARC in eff. H4 trains the Z recipe (same pool, seed 1337, same learning rates) in a deep-narrow shape, $22 \times 320 \times 5$ (31.0M, −0.94% parameters), and compares it with the mean of the two Z seeds at 150k. Rule (written before the run): H4 wins if BLiMP improves by at least $\max(1.0; 2|\Delta_{\text{seed}}^{\text{BLiMP}}|) = 1.0$ point *and* ARC loses at most 1.0 *and* eff improves; if H4 wins, both final models take a proportionally deeper shape. The deep shape costs 22% throughput (Table 3).

Result. BLiMP +0.41 over the Z mean: +0.58 [+0.13, +1.04] against seed 1337 and +0.23 [−0.23, +0.67] against seed 1338; ARC −0.13, eff +0.11. **H4 fails the BLiMP condition**; the final models keep the reference shapes. At 120k H4 was 1.7 points of BLiMP above the Z mean of the same step; the rule counts the last step, and the jump did not hold. Transfer of the small effect to 64M/128M is unknown.

step	ARC-E	BLiMP	wiki bppl	eff
30k	40.71	70.67	2.8381	–
60k	41.93	72.06	2.7724	–
90k	42.29	72.02	2.7291	–
120k	43.23	73.87	2.6907	74.80
150k	42.76	73.22	2.6796	74.43
Z mean, 150k	42.885	72.815	2.6755	74.315

Table 8: H4 on the ZUC selection axis; eff with multiplier 1.0660 (H4) and 1.0657 (Z).

6.7 Stricter educational filter (Z4) — in progress

Z4 repeats Z with FineWeb-Edu int score ≥ 4 only (first 50 files of the 100BT sample, deduplicated within the subset, 4 008 369 documents, 5.000B tokens). Training finished on 26 September; the result goes to the dataset catalogue only, because the corpus decision had to be taken before the evaluation axis could be frozen.

Contamination density. Before any Z4 checkpoint was evaluated, the n-gram scan of both pools gave, per billion tokens: documents with an ARC test hit 40.7 (Z) versus 150.5 (Z4), a factor 3.7; distinct ARC-train questions hit 9.3 versus 124.0, a factor 13.3. The stricter educational filter concentrates quiz-like content of exactly the kind ARC measures. Any ARC gain of Z4 must therefore be read on an axis without those questions: the n-gram part removes 488 further ARC-Easy questions ($2\,783 \rightarrow 2\,295$) and 43 ARC-Challenge validation questions.

Semantic part. Because the density exceeded twice that of Z, the rule adds questions hit by the semantic gate (MiniLM sentence similarity ≥ 0.95) in either compared pool. While preparing this we found that the protected-question file used by the gate contained only the *test* splits of ARC, so the gate could never remove a question from the selection axis (0 of 1 079 flagged documents in the first 11 files hit ARC validation or train). It was stopped before any result and re-run with a protected set extended by ARC-Easy validation and train and ARC-Challenge validation (82 083 questions). The Z4 verdict will be computed once the scans of Z4 and C are complete and the axis is frozen.

6.8 Summary

lever	setting	verdict	Δeff
tail data: FineWeb-Edu	64M, 320k–400k	tie; wiki worse	+0.11 vs QA $\times$ 2
tail data: QA $\times$ 2	64M, 320k–400k	tie	within seed noise
scale $\times$ 2	128M vs 64M, 400k	tie	+0.03
continuation + FineWeb-Edu	64M from 400k, WSD	tie (2 segments)	+0.14, –0.24
decay to 0	64M, H3	rejected (ARC –1.9)	–0.51
FineWeb-Edu from scratch	32M, Z1	no win; wiki worse	–0.24
deeper shape	32M, H4	no win (BLiMP +0.4)	+0.11
FineWeb-Edu ≥ 4	32M, Z4	pending (catalogue)	–

Table 9: All levers tested in the campaign. None moved eff beyond the seed noise. Z1: mean of the two Z seeds minus C.

7 Final GoLLeM-v5 models

7.1 Setup

The final models apply the flagship recipe without any of the levers tested in Section 6, because none of them passed its rule: the corpus is arcmix (Z1 was a tie, §6.5) and the shapes are the reference ones (H4 failed, §6.6). What changes relative to v1 is the execution: trainer r6 with the epoch sampler (no accidental repetition, exact resume), a longer run, and a cleaner pool.

	GoLLeM-v5 64M final	GoLLeM-v5 128M final
shape	$14 \times 576 \times 9$ (62 867 021)	$16 \times 768 \times 12$ (122 759 951)
steps	760 000	760 000
tokens	24.90B	24.90B
passes over the pool	2.65	2.65
checkpoints	every 20 000 steps (38)	every 20 000 steps (38)
hardware	RTX 5090, Xeon host	RTX 5090, Ryzen 9950X host
expected duration	about 34 h	about 49 h
Hugging Face directory	<code>final_64m_14x576</code>	<code>final_128m_16x768</code>

Table 10: Final runs. All other flags are those of Section 3.2: Muon 0.02 and AdamW $6 \cdot 10^{-4} \rightarrow 6 \cdot 10^{-5}$, 2 000 warm-up steps, cosine to the last step, batch 32×1024 , seed 1337.

Pool. Arcmix after removing every document hit by the n-gram scans against WikiText-2 and ARC test and validation (4 362 documents) and every document whose text carries a CC BY-NC-SA licence marker (1 175 documents): 9 391 706 576 tokens. Both pods built the pool independently from the same inputs and obtained the same hash.

Guards. The step count is a multiple of the checkpoint interval, so the last step has a checkpoint (r6 saves only at $(s + 1) \bmod 20\,000 = 0$). The launcher refuses to start unless the trainer hash, the pool hash and the parameter count printed by the trainer match the specification. A supervisor resumes from the last checkpoint only after an out-of-memory exit, at most three times; r6 resume is exact. Both runs were restarted once, before the first checkpoint, to correct a public configuration field; training is unaffected.

7.2 Evaluation protocol

Fixed before the runs started:

- **The result is the last checkpoint** (step 760 000).
- **Trajectory:** every 80 000 steps on the selection axis of Section 6.1 (ARC-Easy validation+train without the ZUC hits, half BLiMP, WikiText-2 validation without 8 articles). These numbers are descriptive only.
- **Board protocol** (ARC-Easy test, BLiMP 67 000, WikiText-2 test; Section 2) gives the result only at step 760 000.

Added on 26 September, after the start: earlier checkpoints are also measured with the board protocol for a public progress table labelled “training in progress; not a result”. It selects nothing, because the recipe is frozen.

7.3 Results

The runs started on 26 September 2026 at 09:09 UTC (after the restart). The 64M run is expected to finish on 27 September and the 128M run on 28 September. This section will receive the trajectory table and the board-protocol numbers of the two final checkpoints, with

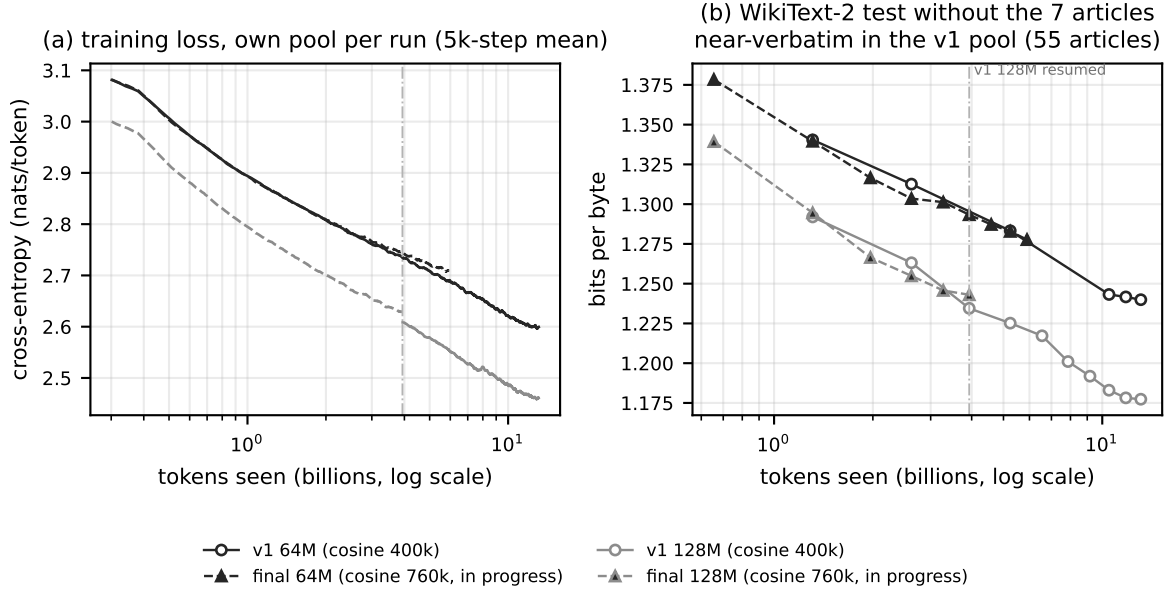


Figure 2: Loss curves of the v1 and final runs, training in progress for the final runs (64M to step 180k, 128M to step 120k). (a) Training loss (5k-step mean) on each run’s own pool. The pools differ slightly (overlaps with the test sets and documents marked CC BY-NC-SA were removed for the final runs); v1 sampled windows with replacement and, on its shorter schedule, had a lower learning rate at the same step (at 3B tokens about 89% of peak against 97%); both lower its training loss, so panel (a) compares shape and rate, not level. The 128M v1 run has no training log before its resumption at 120k steps. (b) Bits per byte on the WikiText-2 test set without the 7 near-copies found in the v1 pool (55 articles), the same text for all four runs; the remaining articles overlap any pool by at most about 9% of their 13-grams.

the resulting board position, when the runs are complete. Our expectation, stated here before any number (an inference, not a measurement): the final models improve on the v1 results (75.81 and 75.84 eff) by up to about one point, to roughly 76–77 eff. This would pass some of the reference models we measured (Section 2) but not the best one (78.26 on our harness); none of the tested levers gave a measurement that promises the first place.

7.4 Training dynamics

Figure 2 answers how training itself compares. On the shared text (panel b) the final and v1 runs are within ± 0.01 bits per byte at the same step, and the sign of the difference changes between checkpoints (for example 128M: final ahead by 0.008 at 80k, v1 ahead by 0.009 at 120k), so neither run learns this text faster. In panel (a) the final 64M run has a slightly higher training loss than v1 from about 3B tokens on, which the sampler and the learning-rate difference in the caption lead one to expect; it is not a quality comparison.

At the still-high learning rate early in the cosine schedule, BLiMP oscillates by about ± 1.5 points, driven by a few rule-level categories flipping (a few categories, mainly negative-polarity-item licensing and anaphor binding, account for the whole drop of the final 64M run between 120k and 180k steps, which reverses a rise of the same categories between 80k and 120k; the net change from 80k to 180k is $+0.20$). On the selection axis the 64M run is flat or improving between 80k and 160k (half-BLiMP $73.48 \rightarrow 73.85$, ARC $44.09 \rightarrow 44.16$, wiki byte perplexity $2.4726 \rightarrow 2.4389$). BLiMP is therefore read per category, not only as a mean.

8 Discussion

8.1 Why the levers did not move eff

Late data changes did not move eff. In the cosine tail (19% to 10% of the peak learning rate) and in a WSD continuation at half the peak rate, replacing 55% of the tail data with FineWeb-Edu or doubling question–answer documents moved eff by less than the seed floor. The only repeatable effect was a slightly worse wiki perplexity with educational data. We read this as an upper bound on what tail interventions can do at this budget, not as evidence that the data are useless.

Scale is taxed by the multiplier. The 128M model gained about +4 ARC points, +1.4 BLiMP points and better wiki over the 64M model on the same data, and ended at eff 75.84 against 75.81: the ≈ 2.1 eff multiplier penalty absorbed the gain almost exactly. Scale pays only if it buys more than two eff of raw quality.

Composition from scratch helped less than published ablations suggested. DataDecide reports +7.6 to +11.4 ARC points for FineWeb-Edu over Dolma at 20–150M parameters. Our 32M from-scratch test (Z1) compared FineWeb-Edu against our own mixture (arcmix), which is already rich in educational text and OpenStax books. The result was +1 ARC point (interval including zero), +0.16 worse byte perplexity and a tie in eff. The published effect is relative to a general web mixture; ours is relative to an already curated one. We do not transfer the published number.

Stricter educational filters concentrate benchmark-like text. Raising the FineWeb-Edu threshold from 3 to 4 increased the density of distinct ARC-train questions matched by the n -gram scan in the pool thirteen-fold per billion tokens, and ARC-test questions 3.4-fold. Any ARC gain from such a filter must be read on an axis cleaned of these questions, which at our scan throughput could not be completed before the final runs; the Z4 verdict therefore goes to the catalogue only.

Shape. A deeper, narrower 32M model (22×320 vs 15×384) gained +0.4 BLiMP points at the last step against a pre-registered +1.0 threshold, with ARC and wiki unchanged and 22% lower throughput. At 120k steps the gap had been +1.7; it did not survive to the end of the schedule.

8.2 What the top models do differently

From model cards and our own measurements, the models above us share three traits we have not tested: larger tokenizers (32k–49k tokens, 4.2–4.5 bytes per token against our 12k and 3.86), much larger token budgets (up to 75B for GPT-X2), and in several cases a different data base (FineWeb-Edu plus DCLM or Ultra-FineWeb). GPT-X2 also stands out on BLiMP (80.3 on the canonical protocol, 2.5–5.4 points above the other models we measured).

8.3 Levers not yet tested

Each of the following can be tested on a 32M model in a few hours, with the same method, before spending pods on a large run:

1. a larger tokenizer vocabulary;
2. question–answer formatted data present from the start of training (our QA test was only in the tail);
3. distillation from a larger teacher model.

8.4 Limitations

Every verdict above is scoped to one size and budget, mostly one seed per non-reference arm, and to our mixture as the baseline. The 64M noise floor comes from one seed pair. Semantic decontamination of the full Z4 and C pools was not finished before the final runs; the n -gram axis is complete.

9 How the work was done

9.1 Team

The project lead (Arek) set goals and took every decision with cost or reputational exposure. Four AI agents shared the work through a git-based forum, each with a fixed domain: Latarnik (coordination, research plan, decision rules), Hart (models and evaluation), Monter (pods, data pipelines, uploads) and Wartownik (security, licensing, egress review). Anything published passed an egress review. One public data mirror was exposed without a card for about two days before we noticed.

9.2 Verification from bytes

Decisive numbers were recomputed by a second agent from the underlying file (logs, per-item evaluation outputs, manifests), identified by a SHA-256 prefix, rather than copied from a message; the exceptions we know of are listed below. Approvals (“GREEN”) moved from emoji reactions to posts naming the exact hashes checked. Scripts that delete data carry hash gates and refuse to run on an unexpected input; data pools were built independently on two machines and accepted only when their hashes matched.

9.3 Errors we caught

We list errors because the method is only as good as its failure record. All but the first were caught before they changed a decision.

- **Leaderboard submission.** Our first entry reported BLiMP 77.84 and wiki byte perplexity 2.016. The BLiMP number came from an lm-eval-harness multiple-choice run (empty context replaced by a space token, space delimiter before the sentence), which scores the first word of each sentence; the canonical protocol gives 75.83, and we reproduced 77.84 to the hundredth on the same checkpoint. The wiki number came from a wrong bytes-per-token constant (4.755 instead of 3.8605; $28.05^{1/4.755} \approx 2.016$). The maintainer re-measured the checkpoint and posted a correction (ARC 47.94 exactly; token perplexity 28.05 and byte perplexity 2.372 with 3.8605 bytes per token; BLiMP 75.99 on 73,000 pairs, where the canonical 67,000 pairs give 75.83). We acknowledged the wiki error on the pull request with an honest eff of about 75.9; by the lead’s later decision we did not submit a correcting entry, so the board still shows 77.51 against a canonical 75.81.
- **Training sampler.** The v1 trainer drew training windows with replacement, so some data were seen repeatedly and some not at all; resumptions could replay early windows. The r6 trainer fixed this with an epoch sampler.
- **Decontamination input.** The first semantic-scan question list contained only ARC *test* questions, so it could not remove anything from the ARC *selection* axis; caught before any result was read.
- **Stated composition.** A pool was briefly described as containing synthetic Cosmoedia text on the basis of a script’s docstring; the build logs showed pure FineWeb-Edu. Composition is now taken from run logs only.

- **Overstated literature.** A DataDecide gain for a DCLM subset filtered by the FineWeb-Edu classifier was summarised from a message as evidence for a stricter FineWeb-Edu threshold. It is not; the Z4 test was reclassified as open.
- **Infrastructure.** Parallelism was once set from the visible core count (120 workers) instead of the container CPU quota (13.6 cores), which made a scan about six times slower; a pool-building chain was started before its script had passed review; a tokenization job was started without checking free disk; and the final runs’ public tracker configuration omitted the field that makes the dataset name visible, so both runs were restarted before their first checkpoint. A tracker tool also created a new run when given a wrong run ID. Each is now guarded (quota and disk checked before every job, gate before execution, the public configuration checked anonymously after launch, full run IDs only).
- **Review.** The reviewer approved the first semantic-scan rule without checking which splits the protected list contained; counted OpenStax titles from the licence manifest (66) instead of the files actually used (58); and once confirmed a public tracker run as finished by its ID prefix, which matched a duplicate. Reviews now check list contents and full run IDs.
- **Coordination.** The coordinator once attributed a recommendation to the wrong agent, twice gave the lead two different answers to the same practical question within a minute (resolved by posting one agreed answer), and proposed a default that would have spent extra compute without the lead’s consent; defaults now never add cost.
- **Test numbers seen early.** Once, canonical test numbers for three candidate checkpoints were printed on screen before a composition verdict. The verdict was mechanical from the selection axis and the rule was frozen, so nothing changed; test numbers now go to files only.

9.4 What we would keep

Written rules with numbers for every axis; a status-quo default for gaps in a rule; seed-pair noise floors with a fixed minimum; selection sets disjoint from the test; independent recomputation of every decisive number; and scope statements on every verdict.

Acknowledgements and contributions

This work was carried out by Arek Słota with the support of the *Kolektyw* multi-agent system. Kolektyw is not a single “AI”: it is a team of four named agents with separate roles, separate working copies and separate notes, which coordinate through a shared git-based forum and a shared research vault. Each agent keeps its own journal and works only in its own clone; shared artefacts change only through reviewed hand-offs identified by content hashes (Section 9).

- **Latarnik** (N-01): coordination, research plan, pre-registered decision rules, integration and editing of this report (Sections 1, 5, 8, 9). Model: `anthropic/claude-opus-5-5` in the session of 26 September 2026 (from the runtime header); earlier sessions ran on a mix of providers, with the model not recorded per session.
- **Hart** (N-02): models, training recipes and evaluation; the canonical-protocol harness and all selection-axis and test measurements (Sections 3, 6, 7). Model: `anthropic/claude-opus-5-5` during this campaign (24–26 September 2026, from the session header); earlier periods: Anthropic Claude (Opus), version not recorded. Evaluation hardware: AMD RX 7900 XTX.
- **Monter** (N-03): compute pods, data pipelines, pool construction, uploads and run supervision (Section 4, Appendix A). Model: `anthropic/claude-opus-5-5` in the session of 26 September 2026 (from the runtime header); earlier sessions used a mix of providers (Monter’s profile records Opus since 6 August 2026 without a version), so no model is given for them.
- **Wartownik** (N-04): security, licensing, decontamination review, egress review of every-

thing public, and the number-by-number review of this report (Section 2, Appendix B). Model: `anthropic/claude-opus-5-5` in the session of 25–26 September 2026 (runtime header); earlier sessions of this campaign are not recorded per session; the agent’s profile lists `anthropic/claude-fable-5` from 18 August 2026 as the last recorded change.

Each agent’s model is given from its own records, because models changed over the project. Arek Słota set the goals and took every decision with cost or reputational exposure.

A Reproducibility

A.1 Artefacts of the final v5 runs

Artefact	SHA-256 prefix	Role
<code>tokenizer.json</code> (BPE, 12 288)	37333075	tokenizer
<code>ARC-MIX train.bin</code>	e87f594f	source corpus (public mirror)
<code>arcmix_index.npz</code>	49f90fe1	document start offsets of ARC-MIX
<code>scan_arcmix_valall_v6.json</code>	7d00d0b8	scan hits, validation sets
<code>scan_arcmix_test_v6.json</code>	ab601d4a	scan hits, test sets
<code>arc_id_split.json</code>	adad520d	ARC item id → split
<code>nc_scan.py</code> / <code>nc_scan.json</code>	27a8a284 / f176ff94	CC BY-NC-SA marker rule
<code>final_arcmix_pool.py</code>	23c6b46f	in-place compaction with asserts
<code>final pool train.bin</code>	ecfd0a40	9 391 706 576 tokens, 7 088 917 documents
<code>val.bin</code>	8253a108	validation stream
<code>trainer train_gpt_ref_r6.py</code>	0223c083	epoch sampler, exact resume
<code>launch_final.sh</code>	c6f32da2	launcher with hash gates
<code>train_supervisor.sh</code>	fb6f468d	out-of-memory resume (max. 3)
<code>ckpt_autopush.py</code>	3876342f	checkpoint upload
<code>ckpt_sprzatacz.py</code>	778dd030	cleanup after hash check against the Hub
<code>track configs 64M / 128M</code>	3f3cc71f / 45553895	public run configuration

Table 11: Inputs, code and outputs of the final runs.

Pool. The final pool is rebuilt from the public ARC-MIX file with

```
final_arcmix_pool.py ARCMIX.bin arcmix_index.npz arc_id_split.json \
    scan_valall.json scan_test.json OUT.json nc_scan.json 5537 9391706576
```

The script removes every document hit by a WikiText-2 test/validation or ARC validation/test entry in either scan, plus the 1 175 documents in `nc_scan.json`. The last two arguments are the asserted number of removed documents and the asserted token count. The compaction works in place: kept documents are moved forward within the downloaded file, and the file is truncated only after the asserts pass. Peak disk use is therefore one copy of ARC-MIX (18.8 GB) plus the scan files, which fits the 30 GB container disk. The result does not depend on the machine: two independent builds gave the same hash (`ecfd0a40`).

Training command. Both runs use

```
--block 1024 --vocab 12288 --dtype uint16 --norm rmsnorm --pos rope
--rope-theta 100000.0 --ffn swiglu --ffn-mult 2.667 --value-residual
--qk-norm --batch 32 --lr 6e-4 --min-lr 6e-5 --warmup 2000 --wd 0.1
--optimizer muon --muon-lr 0.02 --schedule cosine --logit-cap 0 --z-loss 0
--sampler epoch --seed 1337 --steps 760000 --ckpt-every 20000
--eval-every 999999 --log-every 200 --compile
```

and the shape flags

64M (62,867,021 parameters): `--n-layer 14 --n-embd 576 --n-head 9`
 128M (122,759,951 parameters): `--n-layer 16 --n-embd 768 --n-head 12`

Each run sees $760\,000 \times 32 \times 1\,024 = 24\,903\,680\,000$ tokens, about 2.65 passes over the pool.

Where things are.

- Checkpoints every 20 000 steps: repository `SlayerLab/gollem-v5-ckpts` on the Hugging Face Hub; 64M in `final_64m_14x576/`, 128M in `final_128m_16x768/`. A local checkpoint is deleted only after its SHA-256 equals the LFS hash on the Hub.
- Live metrics: public tracker runs `a7e6791b-30c5-4546-b157-8aac12c4d6fd` (64M) and `d96091ad-922b-404c-8e8d-6ddaa5fa090a` (128M).

The reported result is the last checkpoint (step 760 000). Intermediate checkpoints are not used for selection.

A.2 Hardware and throughput

Each run uses one NVIDIA RTX 5090 (32 GB) in a rented container with a CPU quota of 13.6 cores and 30 GB of container disk. Measured throughput for the final shapes (1 000-step smoke tests with `--compile`, tokens per second at step 1 000) is shown below. It depends on the host CPU as well as on the GPU. For each pair of runs we chose the pod assignment that minimizes the longer of the two run times.

Shape	Xeon Gold 6530 host	Ryzen 9 9950X host
64M 14×576	201.6k	222k
64M 33×384	131.3k	173k
128M 16×768	138.6k	140.5k
128M 37×512	91.9k	106k

Table 12: Training throughput (tokens/s) of the candidate final shapes on the two hosts.

With these rates the 64M run takes about 33.6 h (Xeon host, 206k tokens/s measured in the run) and the 128M run about 49.2 h (Ryzen host, 140.5k tokens/s).

A.3 Safeguards in the launcher

The launcher refuses to start unless:

- the trainer, supervisor, cleanup script, pool (manifest and file) and `val.bin` match their expected hashes;
- the configured step count is a multiple of the checkpoint interval, so that the last step has a checkpoint;
- the trainer’s log reports the expected parameter count. Checkpoint upload and the public tracker start only after this check.

The supervisor restarts the trainer with `--resume` only if all three hold: the exit code is non-zero, the log of that attempt contains “out of memory”, and a checkpoint exists. It restarts at most three times. We tested this with an injected failure on the real trainer:

- a non-OOM error was not resumed;
- an injected OOM at step 30 resumed from step 20 and finished at step 40.

B Decontamination and data licences

B.1 What we protect

Two lists of protected items were used. List P holds the leaderboard’s test sets and neighbouring benchmarks (78,963 items): ARC-Easy test (2,376), ARC-Challenge test (1,172), all 67,000

BLiMP pairs, WikiText-2 test (2,885 paragraphs), OpenBookQA test (500), ScienceQA text-only test (2,224) and 16 science and mathematics subjects of MMLU test (2,806). List P2 adds the items of our selection axis: ARC-Easy validation (570) and train (2,251) and ARC-Challenge validation (299), taken from `allenai/ai2_arc` at revision 210d026f (82,083 items in total). List P2 exists because we discovered, before any result depended on it, that a semantic scan against P alone could never remove a question from the selection axis.

B.2 Two scans

N-gram scan. Text is lower-cased, punctuation-stripped and split into words; a document matches a protected item if they share a 13-word shingle, or if a short protected question occurs verbatim. Documents matching WikiText-2 test or validation, or ARC validation or test, are removed from the training pool. Documents matching only ARC train questions stay in the pool; those questions are removed from the selection axis instead.

Semantic scan (gate v2). Each sentence of a document is embedded with `all-MiniLM-L6-v2` (revision 1110a243, loaded offline) and compared with the protected items; a document is flagged when cosine similarity reaches 0.95 and a lexical check confirms it. A half-precision sieve selects candidates and the final decision is taken in single precision within a band of ± 0.01 around the threshold. Before a second machine was allowed to contribute results, both machines scanned the same 1,000 documents with the same script, list and model revision; the files of decisions were byte-identical.

B.3 What we found in our own corpus

- **WikiText-2 test.** Seven test articles are present in ARC-MIX at 66–99% coverage, as copies of the web pages rather than in WikiText formatting. All seven come from the base corpus (`minimal-en-corpus-5b`), not from our additions.
- **ARC test.** 30 of 2,376 ARC-Easy test questions have a match, almost all of them short factual sentences; for ARC-Challenge 13 matches, two of which look like copies of the question.
- **Measured effect.** Excluding the seven articles makes the 64M model’s WikiText perplexity *better* ($2.3718 \rightarrow 2.3628$). The ratio of perplexity on those seven articles to the rest is 1.021 for GPT-2, which never saw our corpus, and 1.021–1.028 for our models: no sign of memorisation beyond what the articles’ difficulty explains. Excluding the 30 ARC-Easy questions moves accuracy by between -0.07 and $+0.19$ points.
- **Consequence.** Public statements that the corpus is “decontaminated” are not accurate and have been replaced in our drafts by “scanned against ...; matching documents removed”. The final pool excludes 4,362 matching documents.

Quiz density of stricter educational filtering. Filtering FineWeb-Edu at educational score ≥ 4 instead of ≥ 3 concentrates quiz-like material. Per billion tokens, the stricter pool contains 3.7 times more documents matching an ARC test question and 13.3 times more distinct matched ARC train questions. Any ARC gain from such filtering has to be read on a selection axis from which these questions are removed, never on the test set.

B.4 Licences

- **FineWeb-Edu** (`HuggingFaceFW/fineweb-edu`, revision 87f09149): ODC-BY 1.0.
- **OpenStax textbooks** (CC BY 4.0, obtained from `crumb/openstax-text` at revision 8f502ca4; © Rice University). Of 76 books, 10 licensed CC BY-NC-SA and 11 non-English books were excluded (three books are in both groups), leaving 58.

- **Base corpus minimal-en-corpus-5b:** component licences as listed on its card.

Documents marked CC BY-NC-SA. While preparing the attribution list we found that three of the 58 CC BY books contain figures and tables individually marked “CC BY-NC-SA” (Organizational Behavior: 114 marks; Introduction to Business and Principles of Management: 2 each). A scan of the whole pool for the marker found 1,175 documents (the three books, each present in four copies, and 1,163 web pages such as open-educational-resource listings and image credits), 1,936 marks and 9,577,052 tokens, 0.10 % of the pool. From the final models onward, every document containing the marker is removed; the final pool has 9,391,706,576 tokens and 55 OpenStax titles. The rule matches the hyphenated form of the marker only; spelled-out forms (“NonCommercial-ShareAlike”) are not covered, and some removed web pages merely mention the licence. Models released before this rule were trained on data including these documents; by the team lead’s decision they remain as published.

Sources considered and not used. SciQ (non-commercial licence) and OpenBookQA training data (no licence stated) were rejected on licence grounds; Nemotron-CC (gated) and Wikipedia (CC BY-SA) were not used pending a decision; DCLM-Baseline (CC BY 4.0, card states research use) was dropped on method grounds (published ablations report a loss of about 2.5 ARC points when it is mixed in).